

UC
UNIVERSIDAD DE CANTABRIA

**Tecnologías de Desarrollo de Aplicaciones Empresariales sobre
Internet**

Máster en Ingeniería Informática

Documento de Arquitectura - Polaflix

Justificación de Estrategias de Herencia e Identificadores

Autor: Miguel Martín
Fecha: 9 de junio de 2026

1. Estrategias de Herencia

1.1. Decisión: No utilizar herencia

Tras analizar el dominio de Polaflix, se ha tomado la decisión arquitectónica de no utilizar ninguna estrategia de herencia en el mapeo objeto-relacional.

Justificación: Las entidades principales del sistema (*Usuario*, *Serie*, *Temporada*, *Capitulo*, *Persona*, *Factura*, *Visualizacion*, *RegistroSerieUsuario*) representan conceptos independientes. No existe una necesidad semántica ni técnica de compartir comportamiento, estado o atributos comunes a través de una superclase.

Beneficios de esta decisión:

- **Simplicidad:** Cada entidad es autónoma, facilitando la comprensión del modelo.
- **Mantenibilidad:** Los cambios estructurales en una entidad (por ejemplo, añadir campos a *Usuario*) no tienen efectos colaterales en el resto del esquema de base de datos.

2. Estrategia de Identificadores

2.1. Elección: Identificador Numérico (Long) con Generación AUTO

Esta estrategia se ha aplicado a todas las entidades del modelo.

Justificación Técnica: Se utiliza la anotación

```
@GeneratedValue(strategy = GenerationType.AUTO)
```

- **Eleccion:** Se ha elegido este *GenerationType* frente a otros debido a que se delega al ORM la decisión de elegir la mejor estrategia de generación de IDs en función del entorno de la BBDD.
- **Uso del tipo Long (64 bits):** Se ha preferido sobre el clásico *int* previniendo un potencial crecimiento a escala masiva, garantizando que el sistema no se quede sin identificadores disponibles a largo plazo.

Convención de Nomenclatura: Para mantener la consistencia y claridad del código frente a la base de datos, los atributos se nombran siguiendo el patrón *id<NombreEntidad>* (por ejemplo: *idUser*, *idSerie*, *idFactura*).

3. Relaciones

El modelo relacional hace uso de anotaciones JPA para definir las fronteras de los agregados y el ciclo de vida de los datos.

3.1. Relaciones Bidireccionales y Unidireccionales

- **One-To-Many / Many-To-One:** Se utilizan para jerarquías claras. Por ejemplo, *Usuario* → *Factura* o *Serie* → *Temporada* → *Capitulo*. El lado "Many" actúa como propietario de la relación almacenando la clave foránea (*@JoinColumn*).
- **Many-To-Many:** Utilizado en asociaciones de referencia (ej. *Serie* ↔ *Persona* para actores y creadores) o en catálogos personales (*Usuario* ↔ *Serie* para listas de pendientes o terminadas).